

local-sites — State of the Union

For UG. Technical review requested: architecture, current state, what's left, and a Q&A checklist.

Prepared 8 September 2026 · Repo: `dpoore12/local-sites` · Cloudflare account `a3bf1a13d93899d8408b9d1ea94df078` ·

Prepared by Claude (Anthropic), working directly in Dan's terminal on his Mac

One-paragraph version. This is an 87-domain rank-and-rent network built on Cloudflare Pages, one Jinja2 template pipeline, and Telnyx phone tracking. Infrastructure is solid and was hardened this week (a router bug that killed indexing for two weeks is fixed and verified). Google is serving the pages — 16,135 impressions in 28 days — but at an average position around 60 (page six), so clicks are near zero (6 in 28 days) and **realized revenue is \$0**: zero of 87 markets have ever been connected to a paying contractor. The modeled ceiling if every market rented is roughly **\$75,000/mo**. The gap between \$0 and \$75,000 is links and authority, not more pages, not more indexing. Four new test markets went live tonight in a niche (mobile mechanic / towing in small-to-mid cities) chosen specifically because the link bar to rank is 5–10x lower than the legal/garage-door markets that make up the original 83. That's the whole strategic bet right now, and it needs your eyes on the parts of it that are pure engineering risk, listed in §7.

1. What this business actually is

Model: buy a domain, publish a city+service page cluster that reads as a real local business's website, put a real tracking phone number on it, rank it in Google against the map pack and organic results, then rent the finished, ranked site to a real local contractor for a flat monthly fee. The contractor gets calls; we get rent. No calls are ever bought or sold across multiple businesses on one number — each site/number pair is single-tenant, which is the load-bearing legal fact behind the whole model (see `DISCLAIMERS.md`, §7.4 below).

Why rank-and-rent instead of running the businesses ourselves: the contractor already has trucks, licenses, insurance, and a crew. We are selling distribution (page-one presence + a ringing phone), not fulfillment.

Current inventory: 87 domains, all one Cloudflare Pages project, all one Jinja2 build pipeline, all wired to real Telnyx numbers today.

2. Architecture

2.1 Hosting: one Pages project, a hostname router

Cloudflare caps this account at ~22 separate Pages projects. At 87 domains that cap is hit fast, so all 87 sites live in **one** project (local-sites, subdomain local-sites-5d8.pages.dev), and a Cloudflare Pages Function — router/_worker.js — reads the incoming Host header and serves /<host><path> out of the deploy's own file tree. Every domain is a CNAME to the same Pages subdomain; the worker does the multi-tenant split. This is the single piece of code every one of the 87 domains depends on simultaneously — see §7.1.

The same worker also does: `www.` → apex 301, and HTTP Basic Auth (fail-closed if the password env var is missing) in front of the lead-desk paths on each site.

2.2 The redirect bug (fixed, verified this week)

Cloudflare Pages' own internal handling of a trailing-slash-less URL issues a 308 whose Location header leaked the router's internal /<host>/... path back to the client. The client followed it, got a doubled path, and 404'd. This affected **every slash-less URL on all 83 original domains** from 24 Aug until it was found and fixed on 7 Sep — roughly two weeks where a real share of inbound and re-crawl traffic hit a 404 instead of a page. Fix: an `unprefix()` call before the router's own path lookup, stripping the leaked host segment out of any Location header before it goes out. Deployed and confirmed live.

Why this matters for QA: this bug existed for two weeks in production, on every domain, and nobody noticed until a manual audit. There is currently no automated check that would catch a regression like this the same day it ships. See the monitoring ask in §6.3.

2.3 Build pipeline: `template/build.py`

Mechanism	What it does
<code>SHINGLE = 15</code>	Cross-site duplicate-content gate. Build fails if any 15 consecutive words in a site's copy match any other site's copy, network-wide. This is what stops the whole portfolio from reading like one templated mail-merge to Google.
Word-count bands	<code>SERVICE_WORDS</code> (900–2100), <code>PRICING_WORDS</code> (900–2100), <code>LOCATION_WORDS</code> (740–1100), <code>QUESTION_WORDS</code> (700–1150). Each page type has a hard floor and ceiling; the builder refuses to emit a page outside its band.
<code>--live</code> flag	Without it, every page ships <code>noindex</code> , <code>nofollow</code> plus a robots.txt disallow — the default-safe state for anything mid-build. With it, the site is fully indexable. This is exactly the switch that was flipped for the 4 new markets tonight, only after real photos replaced the placeholders.

<code>--check-only</code>	Runs every gate above without writing output — the pre-flight check used before every deploy.
phase (site.json)	1 = bare 9-page site (home/about/contact/services + symptom pages). 2 = full 25-page expansion (adds per-service pages, pricing with sourced cost tables, and optionally per-town location pages). All 87 site.json files are now phase 2 , but only 34 of 87 have actually been built out past 9 pages (see §3 for the real count).
Sourced facts requirement	Every site.json needs ≥ 2 –3 <code>local_facts</code> entries, each with a real, checkable source URL, before the build will pass. Pricing pages require 4–8 cost rows, each with 2–4 real sources. This is a content-integrity floor, not a suggestion — the builder hard-fails without it.

`template/LOCKED.md` freezes `base.html`, `index.html`, `inner.html`, and `assets/theme.css` as a shared design system; `service.html`, `pricing.html`, `location.html`, and `build.py` itself are the editable surface. This is the whole governance model for 87 sites sharing one look.

QA gap: none of the gates above (SHINGLE, word bands, sourced-facts requirement) have unit tests. Correctness is verified by running the real build and reading its printed output. At 87+ sites this is already slow to eyeball and will not scale past a few hundred.

2.4 Deploy mechanism (rebuilt this week, no wrangler on this machine)

Dan's Mac has no Homebrew, no sudo, and no wrangler installed. The historical deploy path (`host_all.py` / `deploy_sites.py`) assumed a sandboxed environment with wrangler and a credential-proxy quirk that doesn't exist here. This week's sessions built a Mac-native replacement, now committed as `live_manifest_mac.py` and `deploy_add_mac.py` at the repo root:

1. `live_manifest_mac.py` re-derives the **true live manifest** by fetching every URL in every site's own `sitemap.xml` plus its known asset paths, and hashing each response body the same way Cloudflare Pages does internally — `blake3(base64(content) + file_extension)`, truncated to 32 hex characters. This was necessary because the last checked-in `data/manifest.json` was stale (23 Aug) and diverged from the live site in ways nobody had tracked.
2. `deploy_add_mac.py` takes that reconstructed manifest, adds any new site's `dist/` output, asks Cloudflare's asset API which content hashes it doesn't already have, uploads only those via an ephemeral upload-token JWT, then POSTs a new deployment referencing the full manifest. It **refuses to run** if any file belonging to an *existing* site would change — a guard against accidentally altering 83 live sites while adding 4 new ones.

Known edge cases found and worked around this week, worth a second look:

- Cloudflare's automatic email-address obfuscation rewrites any HTML page containing a plain email address on every single request (a different encoded blob each time), which makes that page's live bytes un-fetchable byte-exact. Two pages on `cincinnatiipersonalinjurylawyerpros.com` hit this; the obfuscation was reverse-engineered and the original plaintext restored before re-upload. If more pages contain emails, this will recur.

- 8 legacy PNG assets on two sites existed live but were absent from the last manifest entirely (a historical gap, not something introduced this week) — found and folded in during reconstruction.

Ask for UG: this manifest-reconstruction approach works but is fragile by design — it depends on every one of ~2,000 files responding correctly to a fresh fetch at reconstruction time. The correct long-term fix is for `data/manifest.json` to never go stale in the first place: every deploy should end by writing the manifest it just deployed back into the repo (this already happens in `deploy_add_mac.py`, but there's no enforcement that a deploy can't happen any other way). If you want to harden this, that's the seam.

2.5 Phone system: Telnyx + a static-file "call desk" on Vercel

Every market has one Telnyx local number in its own area code, each attached to its own TeXML Application (Telnyx's name for a connection whose call-handling logic is fetched from a URL). That URL points at a plain static XML file hosted on Vercel (`call-desk-xi.vercel.app`), generated by `call-desk/build_xml.py` from `data/markets.json` + each site's `site.json`.

Two states, controlled by one file (`call-desk/routes.json`):

- **Not sold (currently all 87 markets):** call is answered, plays a <4-second greeting naming the business, records a voicemail, hangs up. Nothing is ever recorded except the voicemail itself.
- **Sold:** call rings the contractor's real number for 25 seconds with a whisper announcement ("Call from your <city> <service> website") before connecting, falls back to the same voicemail if unanswered. Flip a market with one command: `python3 flip.py <domain> <number>`.

Call log (`call-desk/call_log.py`) pulls Telnyx call events into a permanent CSV committed to the repo, plus a private, non-indexed web page showing per-market call counts.

Failure-mode gap: no fallback if the Vercel-hosted XML endpoint itself is down when a call comes in — Telnyx would presumably hang up or play its own generic error. Never tested. See §7.2.

2.6 Search Console integration

A desktop OAuth app (`tools/gsc_auth.py`, credentials in `~/gsc/`) drives the Search Console API. Scope was read-only (`webmasters.readonly`) until tonight; it now also holds `webmasters` (write) and `siteverification`, which is what let the 4 new domains get DNS-TXT-verified and sitemap-submitted programmatically in one pass, without touching the Search Console UI.

3. Current state, with real numbers pulled tonight

Metric	Value	Source
Total markets (domains)	87	data/markets.json
— original garage-door / legal / plumbing / etc.	83	
— new this week (towing ×2, mobile mechanic ×2)	4	
Site templates on phase 2 (full expansion contract)	87 of 87	site.json audit, tonight
Sites actually built past the bare 9 pages	34 of 87	live manifest.json audit, tonight
Total built pages network-wide	1,327	same audit
Pages per site — min / median / max	9 / 9 / 25	same audit
Phone numbers wired (own TeXML app each)	87 of 87	call-desk/wiring.json
Markets flipped live to a real contractor	0 of 87	call-desk/routes.json
Tenants signed	0 of 87	every site.json tenant block, audited tonight
Citation packets prepared (not yet submitted)	83	citation-packets/*.md
Cold outreach emails written, CAN-SPAM compliant, not yet sent	405 (189 home-services + 216 legal)	data/emails-*.csv row count
28-day Search Console impressions, all properties	16,135	INDEX-CHECK-2026-09-08.md
28-day clicks, all properties	6	same
Average ranking position	~60 (moved 79→58 over the last 2 weeks, still falling)	same

The single most important line in this document: 0 of 87 markets have ever generated a dollar. Every number above the fold in a pitch deck (the \$71,400–\$75,000/mo modeled figure in §5) is a ceiling if every market rents at its assumed rate. Nothing has been validated end-to-end — not one contractor has been sold, not one call has been routed to a paying customer. That is the actual state of the business tonight.

4. What the niche-sweep found, and why it changed the plan

The original 83 markets are concentrated in garage-door repair, personal-injury/family law, HVAC, and plumbing — categories where the sites that already rank carry **40–90 referring domains**. Building link authority to compete there is slow and expensive, and the network currently has **zero** inbound links doing anything for it.

This week's sweep asked a narrower question: which local-service niche, in which size of city, has a link bar low enough that a well-written, sourced 9-to-25-page site can realistically take a page-one slot without a link-building campaign? Method: DataForSEO `keyword_overview` for volume + average referring domains of ranking pages, cross-checked against Ahrefs (DataForSEO's own average-backlinks figure is provably stale on some queries — a Pueblo, CO test case showed DFS calling a market winnable while Ahrefs's live KD said otherwise), then a live SERP pull and a `bulk_referring_domains` check on the sites actually occupying page one, to rule out national chains.

Service	Cities checked	Passed the gate	What the SERP actually looks like
Mobile mechanic	59	38	Facebook groups, Reddit threads, Wix one-pagers, Ahrefs KD 0–13 in 20+ cities. Ranking sites carry 7–30 referring domains.
Towing (100–300k pop.)	60	26 (DFS gate) / ~10 after Ahrefs cross-check	Similar profile; Richmond VA, Tallahassee, Chattanooga among the strongest.
Towing (50–200k pop., original Kyle-niche sweep)	20	Kalamazoo & Evansville selected as the two test markets	Organic #1 in Kalamazoo carries 17 referring domains; the map-pack incumbent in Evansville carries 9.
Appliance repair	12 (spot check only)	11	Best pass rate of anything tested. Not yet run at full 60-city scale.
Junk removal	12 (spot check only)	10	Second-best pass rate. Not yet run at full scale.
Dumpster rental	12	10 by volume, but crowded	Waste Management, Republic Services, Junk King occupy most page-ones. One more test market at most.

Handyman	12	10	Rejected for now — Washington State requires contractor registration for handyman work; licensing exposure not yet cleared for other states.
Tree removal, pressure washing, informational anything	12 each	0–2	Rejected outright — KD 16–50, or AI Overviews already absorb the click.

Full methodology and every keyword pulled: NICHE-SWEEP-2026-09-07.md and NICHE-SWEEP-2-2026-09-07.md.

5. Revenue model: modeled vs. realized

5.1 Cost side (cheap, not the bottleneck)

Item	Unit cost	87-market total
Domain registration (Cloudflare, at cost)	\$10.46/yr	~\$910/yr
Telnyx local number	\$0.79 once + \$0.79/mo	~\$69/mo recurring
Cloudflare Pages hosting	free at this volume	\$0
Vercel (call-desk static hosting)	free tier	\$0
DataForSEO / Ahrefs API usage (sweep work)	metered per call	non-trivial but not itemized here — ask if you want the exact spend
Optional: citation submission service (top 5 sites, tested idea, not purchased)	~\$250 flat	\$250 if used

Total carrying cost for the entire 87-market network is on the order of **\$70–100/month** plus API metering. The economics only work if traffic converts — infrastructure is not the constraint.

5.2 Revenue side

Modeled rent, original 83 markets (per DOMAINS.md, per-market rent assumed by category/volume)	\$71,400/mo
Modeled rent, 4 new test markets (towing \$1,000/mo × 2, mobile mechanic \$800/mo × 2)	\$3,600/mo
Modeled ceiling, 87 markets, if every one converts	~\$75,000/mo
Illustrative upside if the 36 already-vetted mobile-mechanic markets and ~10 more towing markets are built and rented at the same assumed rates	+\$28,800–\$38,800/mo
Realized revenue, all 87 markets, tonight	\$0

Be clear-eyed reading this: the modeled figures are what the network is worth *if the rank-and-rent thesis works and every market signs a tenant at the assumed rate*. Neither assumption has been tested once. The right next milestone is not "build more markets," it is **get one market to \$1 of real, recurring rent** — ideally one of the 4 new low-competition test markets, since those need the smallest link push to reach page one.

5.3 The map-pack ceiling (a structural constraint, not a bug)

Every local-commercial query, in every city size tested, returns a 3-slot Google local pack that needs a real address and a real Google Business Profile — something a single-tenant, no-fixed-location advertising site cannot legitimately claim. Organic #1 renders 4th on the page, below the pack. Measured AI Overview presence: ~90% of informational queries, ~40% of local-commercial queries, but only 2 of 25 sampled local-commercial queries in this portfolio actually showed one. The revenue ceiling per market is therefore capped below what "#1 organic" sounds like it should be worth, even at full success.

6. What's left to do — technical

6.1 Content

- 53 of 87 markets are still at the bare 9-page contract; the phase-2 (25-page) expansion is written into every `site.json` but not yet executed for most of them.
- Run the full 60-city DataForSEO+Ahrefs sweep on appliance repair and junk removal — both beat towing's pass rate in a 12-city spot check and were never taken to scale.

6.2 Links (the actual bottleneck per §5.2)

- 83 citation packets are written and ready; none have been submitted to any directory.
- 405 cold outreach emails are written and CAN-SPAM compliant (real postal address footer, opt-out path); none have been sent — needs a warmed sending domain (SPF/DKIM/DMARC, 2–4 week ramp at 20–30/day) that doesn't exist yet.

6.3 Monitoring (does not exist today)

- No automated check that all 87 domains 200 OK with the expected title/content. The router bug in §2.2 ran undetected for two weeks; a scheduled synthetic check (cron/GitHub Action hitting each sitemap and validating response) would have caught it same-day.
- No alerting on the phone system — if a Telnyx TeXML app or the Vercel-hosted XML it points to breaks, nobody is notified.

6.4 The actual go-to-market gap

- Zero contractors have been approached to rent a finished, ranked site. This is a sales/ops task more than an engineering one, but it's the one item that turns every number in §5 from modeled to real.

7. Q&A checklist — specifically for you

7.1 — Router correctness. `router/_worker.js` is the one file every one of the 87 domains depends on at once. The known trailing-slash bug is fixed and I verified it live on several domains, but I'd like a second set of eyes on the full hostname-routing + Basic-auth + www-redirect logic, especially edge cases I might not have thought to test (case-sensitivity in hostnames, a domain not present in the deploy, malformed paths).

7.2 — Call-system failure modes. What actually happens to an inbound call if the Vercel-hosted TeXML XML endpoint is down, slow, or returns malformed XML at the moment someone calls? I have not tested this and don't know Telnix's default behavior here. Worth a deliberate failure-mode test before any market goes "sold."

7.3 — Deploy manifest guard. `deploy_add_mac.py` refuses to deploy if any file belonging to an existing site would change. That's the safety property I wanted, but it also means a legitimate future fix to an existing site (a typo, a phone-number update) will get blocked by the same guard that's protecting against accidental damage. Is that the right default, or does it need an explicit "yes, I mean to change this specific file" override?

7.4 — Legal posture, new verticals. `DISCLAIMERS.md` is a thorough single-tenant/no-call-distribution legal memo, but it was written against the legal/HVAC/garage-door verticals. Towing and mobile mechanic carry different state-by-state licensing regimes (see the Michigan/Indiana towing-fee statute research already done for Kalamazoo/Evansville). Worth your read on whether the existing disclaimer posture actually covers these two new verticals, or needs its own addendum.

7.5 — Independently reproduce the ranking finding. The headline technical finding this week is: pages are served (16,135 impressions/28 days) but rank around position 60, producing 6 clicks. I pulled this from the Search Console performance API across all properties. Worth an independent pull + a manual residential-IP search on a couple of queries to sanity-check there's no personalization or API artifact inflating or deflating the number.

7.6 — Secrets hygiene. Credentials currently live as plain files in the home directory (`~/local-sites-token` for Cloudflare, `~/gsc/token.json` for Google, `~/local-sites-leaddesk`). A Vercel personal access token was also pasted directly into a chat session tonight to work around the Claude-Vercel connector never getting proper team-scoped access; my own attempt to auto-revoke it after use did not find a matching token by name, so **that Vercel token should be manually confirmed revoked** at `vercel.com/account/settings/tokens`. Worth a broader pass on whether any of this belongs in a real secrets manager instead of dotfiles, especially once more than one person is touching this repo.

8. Where I could use your help directly

- **Vercel access from Claude never worked properly.** The official Vercel MCP connector reports zero teams even after a full disconnect/reconnect, so tonight's call-desk redeploy needed a manually-issued personal access token instead of the connector. If you have more visibility into Vercel's OAuth/team-scoping, it'd be worth figuring out why the connector can't see `dpoore-6096s-projects`, so future sessions don't need a human to hand-mint a token.
- **No CI/CD exists for any of this.** Every deploy this week ran as ad hoc Python from an interactive session. `live_manifest_mac.py` and `deploy_add_mac.py` are committed and reasonably solid as a starting point — turning them into a real GitHub Action (or at minimum a script with tests and a dry-run mode) would remove the current single point of failure, which is "an AI agent session has to be running for anything to change."
- **Unit tests for `template/build.py`.** The SHINGLE duplicate-content checker and the word-count bands are the entire content-quality gate for the network and currently have zero automated coverage.
- **A real synthetic monitor** for the 87 domains + 87 phone numbers, per §6.3 — this is probably a half-day of work and would have caught the two-week router outage on day one instead of day fourteen.

9. Bottom line

The engineering is in reasonable shape and got materially safer this week: the redirect bug is fixed, four new markets are live end-to-end (domain → DNS → Pages → phone → Search Console verification) on a niche chosen specifically because it needs an order of magnitude less link authority than the original 83. What's unproven is the entire commercial side: zero contractors, zero dollars, and a ranking position (page six, on average) that puts real revenue several weeks to months away even under optimistic assumptions. The highest-leverage next move is not more content or more markets — it's closing the loop on one market, end to end, to prove the model converts a real contractor into real recurring rent. Everything in §7 and §8 is where a second engineer's judgment is worth more than mine right now.